InterviewAce Test Plan

AI-Powered Interview Practice Platform

1. Introduction

This document defines the formal test plan for InterviewAce, an AI-powered interview preparation platform built using React, TypeScript, FastAPI, PostgreSQL, sentence-transformers, Ollama, Docker, and GitHub Actions. The purpose of this plan is to define the scope, environments, resources, schedules, responsibilities, and execution approach used to validate the InterviewAce system across frontend, backend, AI scoring, persistence, integration, and responsiveness workflows.

2. Objectives

The primary objective of testing is to verify that InterviewAce supports the complete job-seeker interview practice workflow from authentication through answer submission, scoring, AI feedback generation, AI coach interaction, and progress/history retrieval. Testing must also ensure that the platform remains stable, secure, deterministic where required, and responsive across desktop and mobile devices.

3. Test Scope

In Scope

• Frontend rendering and interaction behavior
• Backend API validation and JWT enforcement
• Question retrieval and answer submission
• Four-axis scoring and deterministic rubric feedback
• AI Coach chat interaction and fallback behavior
• PostgreSQL persistence and migration integrity
• Responsive layout and usability validation
• CI/CD smoke testing and regression validation

Out of Scope

• Unsupported or undocumented endpoints
• Production-scale load benchmarking beyond defined NFRs
• Real-world email delivery guarantees in CI environments
• Bit-for-bit validation of nondeterministic Ollama responses

4. Test Environment

Testing will be performed using isolated development and CI environments. The frontend uses React + TypeScript with Vite, while the backend uses FastAPI with PostgreSQL and SQLAlchemy ORM. The AI scoring runtime uses sentence-transformers (all-MiniLM-L6-v2), and AI coach functionality is supported by a locally hosted Ollama llama3.2 service. Docker containers are used for environment consistency, while GitHub Actions executes automated CI workflows.

5. Test Levels

Unit Testing
Verifies isolated functions, validators, helpers, frontend components, and scoring utilities.

Integration Testing
Verifies API routes, frontend-to-backend communication, persistence workflows, and AI scoring interactions.

End-to-End Testing
Validates the complete interview practice workflow from login to history retrieval.

Performance Testing
Confirms score response times, frontend rendering performance, and responsiveness targets.

Security Testing
Validates JWT enforcement, password hashing, protected routes, and invalid request handling.

6. Test Deliverables

The following QA deliverables are included in the InterviewAce testing package:
• Test Strategy / Approach document
• Frontend Requirements Traceability Matrix
• Backend Requirements Traceability Matrix
• Frontend Test Case Specification document
• Backend Test Case Specification document
• Test Results Summary document
• Supporting UML, architecture, domain, sequence, deployment, and relationship diagrams

7. Roles and Responsibilities

Frontend QA Responsibilities
Validate rendering, responsiveness, state management, chart display, and frontend interaction workflows.

Backend QA Responsibilities

Validate authentication, scoring, persistence, history retrieval, feedback generation, and API validation behavior.

Integration Responsibilities

Validate end-to-end workflows, database persistence, AI integration behavior, and CI stability.

8. Entry and Exit Criteria

Entry Criteria

- Frontend, backend, and database environments are operational
- Seed data and test accounts are available
- CI pipeline executes successfully
- Requirements and diagrams are stable enough to support traceability

Exit Criteria

- All critical and high-priority tests pass
- No unresolved critical defects remain
- Requirement coverage is documented in RTMs
- End-to-end interview practice flow executes successfully

9. Risks and Mitigations

Risk: Endpoint drift between documentation and implementation
Mitigation: Base tests only on implemented repository routes and continuously update RTMs.

Risk: Nondeterministic AI coach responses
Mitigation: Mock Ollama in CI and validate response structure instead of exact wording.

Risk: Mobile overflow and chart clipping
Mitigation: Run responsive tests at 375 px, 768 px, and desktop widths before release.

Risk: Database migration inconsistency
Mitigation: Execute migration rebuild and persistence-after-restart tests.

10. Test Schedule and Execution Approach

Testing will follow an incremental execution strategy aligned with development phases. Frontend rendering and interaction tests are executed continuously during UI development. Backend API and

persistence tests are executed after endpoint implementation and migration stabilization. End-to-end tests are executed before release/demo milestones. CI smoke tests and lint/build checks execute automatically on every pull request and merge to ensure regression protection.

11. Final Recommendation

InterviewAce should use a hybrid QA strategy combining frontend integration tests, backend API tests, persistence validation, deterministic scoring verification, and lightweight end-to-end smoke testing. Priority should be placed on authentication, question retrieval, /scoring/submit behavior, deterministic score generation, history retrieval, mobile responsiveness, JWT security, and PostgreSQL persistence integrity.

12. QA Coverage Summary

Area	Coverage Focus	Priority
Frontend UI	Rendering, responsiveness, charts, AI Coach UI	High
Backend APIs	Auth, scoring, history, validation	Critical
AI Scoring	Deterministic scoring and feedback	Critical
Database	Persistence and migrations	High
Security	JWT and bcrypt validation	Critical
CI/CD	Regression and smoke validation	High